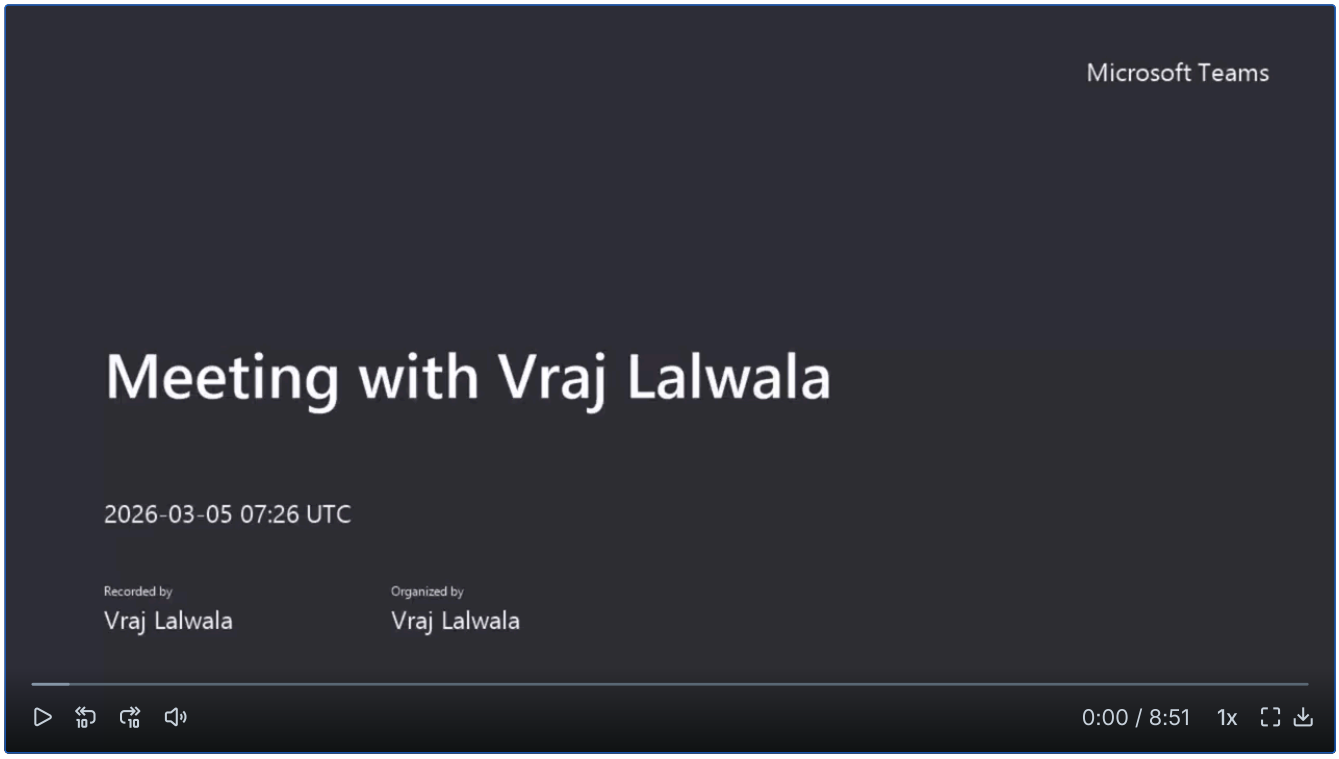
Task-9:-Difference Between Serverless Architecture and Traditional VM-Based Architecture



1. Introduction

This document explains the key operational, architectural, scaling, and cost differences between:

- **Serverless Architecture (AWS Lambda + API Gateway + DynamoDB)**
- **Traditional VM-Based Architecture (EC2 + Flask + Nginx + DynamoDB)**

Both approaches were implemented in this project to demonstrate architectural flexibility and real-world cloud deployment strategies.

---

2. Architecture Overview

2.1 Serverless Architecture (Implemented in Project)

Components Used:

- AWS Lambda (Compute)
- API Gateway (Public API layer)
- DynamoDB (Database)

- ACM (SSL)
- IAM (Security)

Request Flow:

User
→ HTTPS
→ API Gateway
→ Lambda
→ DynamoDB
→ JSON Response

Characteristics:

- No server management
- Fully managed compute
- Event-driven execution
- Automatic scaling

2.2 Traditional VM-Based Architecture (Implemented in Project)

Components Used:

- EC2 Instance (Ubuntu)
- Flask Application
- Nginx Reverse Proxy
- DynamoDB
- Certbot (SSL)
- systemd (Service Management)

Request Flow:

User
→ HTTPS
→ Nginx
→ Flask App (Port 5000)
→ DynamoDB
→ JSON Response

Characteristics:

- Full OS control

- Manual configuration
- Manual scaling
- Continuous uptime billing

---

3. Operational Differences

3.1 Infrastructure Management

Serverless:

- No server provisioning
- No OS patching
- No runtime management
- AWS handles infrastructure

VM-Based:

- Must provision EC2
- Install Python, Flask, Nginx
- Configure firewall (Security Groups)
- Maintain OS updates
- Handle service restarts

In this project:

- Serverless required minimal infrastructure setup.
- VM approach required full Linux configuration and service management.

3.2 Deployment Complexity

Serverless:

- Upload code to Lambda
- Configure API Gateway
- Attach IAM role
- Done

VM-Based:

- Launch EC2

- SSH into instance
- Install dependencies
- Create virtual environment
- Configure systemd
- Configure Nginx
- Configure SSL

VM deployment is significantly more complex.

3.3 Monitoring & Logging

Serverless:

- CloudWatch Logs automatically integrated
- Built-in error tracking

VM-Based:

- journalctl logs
- Nginx logs
- Manual troubleshooting

Serverless offers centralized monitoring by default.

---

4. Scaling Differences

4.1 Automatic Scaling

Serverless:

- Scales automatically per request
- Handles thousands of requests instantly
- No manual intervention required

In this project:
If 1000 users send calculation requests, Lambda scales automatically.

VM-Based:

- Fixed capacity (e.g., t2.micro)

- Limited CPU and RAM
- Requires Auto Scaling Group for scaling
- Manual configuration required

In this project:
If traffic increases heavily, EC2 may crash without scaling configuration.

4.2 Concurrency Handling

Serverless:

- Each request runs in isolated execution environment
- Concurrency limit managed by AWS

VM-Based:

- Flask handles requests sequentially (unless configured with Gunicorn)
- Limited thread handling

4.3 Performance Behavior

Serverless:

- May have cold start latency
- Fast scaling
- No idle cost

VM-Based:

- Always running
- No cold start
- Continuous billing

---

5. Cost Differences

5.1 Serverless Cost Model

- Pay per execution
- Pay per request
- No cost when idle

Ideal for:

- Low to medium traffic
- Event-driven systems

5.2 VM-Based Cost Model

- Pay per uptime
- Charged even when idle
- Fixed instance pricing

Ideal for:

- Constant traffic
- Long-running processes

---

6. Security Differences

Serverless:

- IAM-based permission model
- No direct server exposure
- API Gateway handles SSL
- No SSH access required

VM-Based:

- SSH access required
- Open ports (22, 80, 443)
- Manual SSL setup
- OS-level vulnerabilities possible

Serverless reduces attack surface significantly.

---

7. Maintenance Differences

Serverless:

- No patching

- No OS upgrades
- No infrastructure maintenance

VM-Based:

- Must update OS
- Must update packages
- Must manage systemd services
- Must renew SSL (if not automated)

---

8. Reliability Comparison

Serverless:

- Multi-AZ by default
- Managed by AWS
- High availability built-in

VM-Based:

- Single EC2 instance (single point of failure)
- Requires Load Balancer for HA

In this project:
Serverless is inherently more reliable.

---

9. Flexibility & Control

Serverless:

- Limited runtime customization
- Cannot control underlying OS

VM-Based:

- Full OS access
- Custom configurations possible
- Suitable for complex systems

---

10. Summary Comparison Table

Feature	Serverless	VM-Based
Server Management	None	Full
Scaling	Automatic	Manual
Cost Model	Pay per request	Pay per uptime
Setup Complexity	Low	High
Maintenance	Minimal	High
Security	Managed	Manual
Reliability	High	Depends on config
Performance	Cold start possible	Always warm

---

11. When to Use Which?

Use Serverless When:

- Traffic is unpredictable
- You want low operational overhead
- You want automatic scaling
- You want cost efficiency

Use VM-Based When:

- You need full OS control
- You run long background services
- You need custom runtime configuration
- You require legacy compatibility

---

12. Conclusion

In this PowerOfMath project, both architectures were successfully implemented.

Serverless architecture proved to be:

- More scalable

- More cost-efficient
- Easier to maintain

Traditional VM-based architecture provided:

- Greater flexibility
- Full infrastructure control
- Real-world server management experience

This dual implementation demonstrates a comprehensive understanding of modern cloud-native and traditional deployment models in AWS.